

# KHẢO SÁT CÁC KIẾN TRÚC ENCODER-DECODER HIỆN ĐẠI CHO BÀI TOÁN PHÂN ĐOẠN ẢNH

Thân Thị Hằng, Lê Thị Kiều Trang

Viện CNTT và TT - HVKTQS

## TÓM TẮT

Phân đoạn ảnh (image segmentation) là bài toán gán nhãn ngữ nghĩa cho từng điểm ảnh, đóng vai trò then chốt trong thị giác máy tính. Nghiên cứu này thực hiện *khảo sát so sánh có hệ thống* (systematic comparative study) ba kiến trúc encoder-decoder đại diện cho ba chiến lược kết nối khác nhau — U-Net với skip connections (ResNet-34), SegNet với pooling indices (VGG16-BN) và DeepLabV3+ với ASPP (ResNet-34) — trên hai tập dữ liệu thuộc hai miền ứng dụng: Oxford-IIIT Pet (3 lớp, ảnh tự nhiên) và Kvasir-SEG (nhị phân, ảnh nội soi y tế). Tổng cộng 54 thí nghiệm có kiểm soát (3 mô hình  $\times$  2 datasets  $\times$  3 seeds  $\times$  3 tỷ lệ dữ liệu) tại baseline  $256 \times 256$  cùng 4 thí nghiệm ablation tại  $512 \times 512$  được thực hiện nhằm đánh giá toàn diện về độ chính xác (Mean IoU, Dice), hiệu năng tính toán (latency, FLOPs) và tính bền vững khi thiếu dữ liệu. Kết quả cho thấy U-Net đạt IoU cao nhất tại baseline (0.805 trên OxfordPet, 0.883 trên Kvasir-SEG) nhờ skip connections dày đặc; tuy nhiên, thí nghiệm ablation cho thấy DeepLabV3+ vượt U-Net khi tăng input lên  $512 \times 512$  với output stride 16 (IoU 0.895 vs 0.891 trên Kvasir-SEG), xác nhận ASPP chỉ phát huy khi feature map đủ lớn. SegNet kém hiệu quả nhất cả về accuracy lẫn chi phí tính toán (42.5 GFLOPs, chậm  $2.5 \times$ ). Pretrained encoder giúp duy trì hiệu suất tốt với mức suy giảm IoU dưới 3.5% khi chỉ dùng 25% dữ liệu.

**Từ khóa**— Encoder-decoder, phân đoạn ảnh, U-Net, SegNet, DeepLabV3+, transfer learning, pretrained encoder

## 1. GIỚI THIỆU

Phân đoạn ảnh (image segmentation) là bài toán gán một nhãn ngữ nghĩa cho mỗi điểm ảnh (pixel) trong ảnh đầu vào. Cụ thể, đầu vào là ảnh RGB kích thước  $H \times W \times 3$ , đầu ra là bản đồ phân đoạn (segmentation mask)  $H \times W$  với mỗi giá trị tương ứng một lớp ngữ nghĩa. Bài toán này đóng vai trò nền tảng trong nhiều lĩnh vực ứng dụng: phát hiện polyp trong nội soi đại tràng [1], phân tích ảnh y tế, xe tự lái, nông nghiệp chính xác và viễn thám [2].

Kể từ khi Long et al. [3] đề xuất Fully Convolutional Network (FCN), kiến trúc encoder-decoder đã trở thành lớp phương pháp chủ đạo cho bài toán phân đoạn, trong đó phần *encoder* trích xuất đặc trưng (features) ở nhiều mức trừu tượng hóa với độ phân giải giảm dần, còn phần *decoder*

có nhiệm vụ khôi phục lại độ phân giải ban đầu để tạo ra segmentation mask. Thách thức cốt lõi nằm ở cơ chế *kết nối giữa encoder và decoder*: làm thế nào để decoder tận dụng tối đa thông tin từ encoder nhằm khôi phục các chi tiết không gian (spatial details) đã bị mất qua quá trình downsampling. Ba chiến lược kết nối tiêu biểu là: (1) *Skip connections* — nối trực tiếp feature maps từ encoder sang decoder (U-Net [4]); (2) *Pooling indices* — chỉ truyền vị trí max pooling để hướng dẫn upsampling (SegNet [5]); (3) *Multi-scale context* — sử dụng atrous convolution ở nhiều tỷ lệ để thu thập ngữ cảnh (DeepLabV3+ [6]). Sự khác biệt về cơ chế kết nối dẫn đến các đánh đổi (trade-offs) khác nhau giữa độ chính xác, chi phí tính toán và khả năng khái quát hóa.

Mặc dù đã có nhiều nghiên cứu tổng quan [2] và so sánh riêng lẻ, *ít có công trình nào thực hiện so sánh công bằng cả ba chiến lược kết nối trên cùng điều kiện huấn luyện*, cùng encoder pretrained, và trên nhiều miền dữ liệu. Bài báo này, về bản chất là một *khảo sát so sánh có hệ thống* (systematic comparative study), lấp đầy khoảng trống đó bằng cách thiết kế thí nghiệm có kiểm soát chặt chẽ theo phương pháp hypothesis-driven, nhằm trả lời cho yêu cầu nghiên cứu *Investigating the modern encoder-decoder architectures for image segmentation*.

### 1.1. Câu hỏi nghiên cứu và giả thuyết

Chúng tôi đặt ra ba câu hỏi nghiên cứu (Research Questions) cùng các giả thuyết tương ứng:

**RQ1 — Accuracy:** Trong ba kiến trúc, mô hình nào đạt Mean IoU cao nhất trên mỗi miền dữ liệu?

- H1a:** U-Net vượt trội trên OxfordPet nhờ skip connections giữ thông tin biên.
- H1b:** DeepLabV3+ vượt trội trên Kvasir-SEG nhờ ASPP đa tỷ lệ.

**RQ2 — Efficiency:** Đánh đổi giữa accuracy và chi phí tính toán (latency, memory, FLOPs) ra sao?

- H2:** SegNet sẽ tốn tài nguyên nhất do VGG16 encoder có số lượng FLOPs lớn hơn ResNet-34 và decoder phải reconstruction toàn bộ.

**RQ3 — Data scarcity:** Pretrained encoder giúp duy trì hiệu suất như thế nào khi lượng dữ liệu huấn luyện giảm?

- H3:** Tất cả mô hình sẽ suy giảm IoU dưới 5% khi giảm xuống 25% dữ liệu, nhờ pretrained encoder đã học được

các đặc trưng tổng quát từ ImageNet.

Phần còn lại của bài báo được tổ chức như sau: Mục 2 trình bày chi tiết kiến trúc mô hình và phương pháp huấn luyện; Mục 3 mô tả tập dữ liệu, thiết lập thí nghiệm, và phân tích kết quả theo từng câu hỏi nghiên cứu; Mục 4 đưa ra kết luận, hạn chế và hướng phát triển.

## 2. PHƯƠNG PHÁP

### 2.1. Tổng quan kiến trúc

Cả ba mô hình tuân theo kiến trúc encoder–decoder, trong đó encoder sử dụng trọng số pretrained trên ImageNet [7, 8, 9] để trích xuất đặc trưng đa mức. Sự khác biệt trực quan giữa ba chiến lược kết nối encoder–decoder được minh họa trong Hình 1, và Bảng 1 tóm tắt các đặc trưng chính của từng kiến trúc.

**Bảng 1.** So sánh kiến trúc ba mô hình encoder–decoder

| Mô hình    | Encoder   | Decoder         | Kết nối E-D      |
|------------|-----------|-----------------|------------------|
| U-Net      | ResNet-34 | 5×ConvT2d       | Skip concat      |
| SegNet     | VGG16-BN  | 5×MaxUnpool     | Pooling indices  |
| DeepLabV3+ | ResNet-34 | ASPP + Bilinear | Low-level fusion |

### 2.2. U-Net + ResNet-34

Encoder là ResNet-34 (bỏ các lớp fully-connected và average pooling cuối), tạo ra feature maps tại 5 mức độ phân giải:  $128 \times 128$ ,  $64 \times 64$ ,  $32 \times 32$ ,  $16 \times 16$ ,  $8 \times 8$ . Decoder gồm 5 bậc, mỗi bậc thực hiện: (1) ConvTranspose2d để upsampling  $\times 2$ , (2) *concatenate* feature maps tương ứng từ encoder qua skip connection, (3) hai lớp Conv-BN-ReLU.

Đặc trưng cốt lõi: các *skip connections* thực hiện nối trực tiếp (*concatenate*) giúp truyền *toàn bộ feature maps* từ encoder sang decoder. Điều này giúp decoder tiếp cận cả ngữ nghĩa cấp cao lẫn chi tiết không gian cấp thấp, đặc biệt quan trọng cho phân đoạn có đường biên phức tạp.

### 2.3. SegNet + VGG16-BN

Encoder là VGG16 có Batch Normalization gồm 13 lớp convolution chia thành 5 blocks, mỗi block kết thúc bằng MaxPool2d. Decoder gồm 5 bậc đối xứng; thay vì dùng feature maps từ encoder, SegNet *chỉ lưu và truyền pooling indices* — tức là vị trí của giá trị lớn nhất trong mỗi cửa sổ max pooling — để hướng dẫn MaxUnpool2d trong decoder.

Ưu điểm: tiết kiệm bộ nhớ truyền giữa encoder–decoder (chỉ cần chỉ số thay vì toàn bộ tensor). Nhược điểm: mất hoàn toàn thông tin texture và ngữ cảnh từ encoder, buộc decoder phải tự học lại toàn bộ reconstruction từ các convolution layers riêng.

### 2.4. DeepLabV3+ + ResNet-34

Encoder ResNet-34 trích xuất features ở Stage 4 ( $8 \times 8$ ), sau đó đưa vào module ASPP (Atrous Spatial Pyramid Pooling) gồm 5 nhánh song song:

- Convolution  $1 \times 1$  (không dilation)
- Atrous convolution  $3 \times 3$  với dilation rate = 6, 12, 18
- Global Average Pooling +  $1 \times 1$  convolution

Năm nhánh được concatenate rồi giảm chiều qua  $1 \times 1$  conv. Decoder nhận đầu ra ASPP (upsample  $4 \times$ ), kết hợp với *low-level features* từ Stage 1 của ResNet ( $64 \times 64$ ) qua  $1 \times 1$  conv giảm kênh, rồi upsample  $4 \times$  lần nữa để đạt độ phân giải gốc.

Đặc trưng: thu thập ngữ cảnh đa tỷ lệ (multi-scale context) qua các receptive field khác nhau trong ASPP, đặc biệt hiệu quả cho objects có kích thước biến thiên lớn.

### 2.5. Hàm mất mát

Chúng tôi sử dụng hàm mất mát kết hợp (hybrid loss) giữa Cross-Entropy Loss và Dice Loss [10]:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{Dice}} + (1 - \alpha) \cdot \mathcal{L}_{\text{CE}}, \quad \alpha = 0.5 \quad (1)$$

trong đó Dice Loss giúp xử lý mất cân bằng lớp (class imbalance) — vấn đề nghiêm trọng trên OxfordPet với lớp boundary chỉ chiếm  $\sim 2\%$  diện tích — còn Cross-Entropy cung cấp gradient ổn định cho quá trình tối ưu.

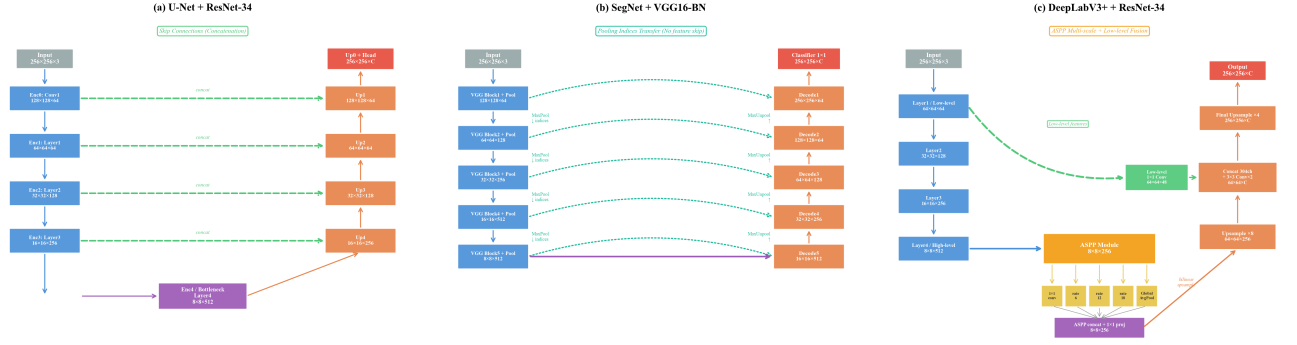
### 2.6. Cấu hình huấn luyện

- **Optimizer:** AdamW [11], learning rate =  $3 \times 10^{-4}$ , weight decay =  $10^{-4}$
- **Scheduler:** CosineAnnealingLR — giảm dần LR theo hàm cosine từ  $3 \times 10^{-4}$  về  $\sim 0$  trong 25 epoch
- **Mixed Precision:** AMP (Automatic Mixed Precision) với GradScaler để tăng tốc trên GPU T4
- **Early Stopping:** Theo dõi val IoU, patience = 7 epoch
- **Batch size:** 8; Input size:  $256 \times 256$

Learning rate  $3 \times 10^{-4}$  được chọn vì đây là giá trị chuẩn cho fine-tuning pretrained models với AdamW, đảm bảo encoder không bị catastrophic forgetting trong khi decoder có đủ gradient để học. Việc sử dụng *cùng một LR* cho tất cả mô hình là lựa chọn *có chủ đích* nhằm đảm bảo biến kiểm soát duy nhất là kiến trúc. Nếu tối ưu LR riêng cho từng model, không thể phân biệt lợi thế đến từ kiến trúc hay từ hyperparameter — do đó fair comparison là yêu cầu cốt lõi.

### 2.7. Tăng cường dữ liệu

Sử dụng Albumentations [12]: lật ngang/dọc ( $p=0.5$ ), xoay ngẫu nhiên, điều chỉnh sáng/tương phản, CoarseDropout, biến dạng đàn hồi và ShiftScaleRotate. Ảnh resize  $256 \times 256$ , chuẩn hóa ImageNet. Val/test chỉ resize và normalize.



**Hình 1.** So sánh trực quan ba chiến lược kết nối encoder-decoder: (a) U-Net với skip connections (concat), (b) SegNet với pooling indices, (c) DeepLabV3+ với ASPP và low-level fusion.

### 3. KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH

#### 3.1. Tập dữ liệu

**Oxford-IIIT Pet [13]:** Gồm 7,349 ảnh tự nhiên chó/mèo với segmentation mask 3 lớp: nền (background), thú cưng (pet) và viền (boundary). Lớp boundary chiếm tỷ lệ rất nhỏ (~2% diện tích), tạo ra bài toán *mất cân bằng lớp nghiêm trọng* (severe class imbalance). Đây là thách thức đặc trưng cho phân đoạn ảnh tự nhiên.

**Kvasir-SEG [1]:** 1,000 ảnh nội soi đại tràng, mask nhị phân (nền, polyp). Kích thước polyp biến thiên lớn, tạo thách thức xử lý đa tỷ lệ. Tập dữ liệu nhỏ hơn nhiều, đặt ra câu hỏi về khả năng khái quát.

Cả hai tập được chia 70%/15%/15% (train/val/test) với phân tầng (stratified), đảm bảo phân phối lớp nhất quán.

#### 3.2. Thiết lập thí nghiệm

Toàn bộ thí nghiệm được thực hiện trên Kaggle T4 GPU (16 GB VRAM). Ma trận thí nghiệm kiểm soát bao gồm:

- **3 mô hình:** U-Net, SegNet, DeepLabV3+
- **2 tập dữ liệu:** OxfordPet, Kvasir-SEG
- **3 random seeds:** 42, 2025, 3407 — đảm bảo tính tái lập (reproducibility)
- **3 tỷ lệ dữ liệu:** 100%, 50%, 25% — khảo sát ảnh hưởng của data scarcity
- **1 learning rate:**  $3 \times 10^{-4}$  — đảm bảo fair comparison

Tổng cộng:  $3 \times 2 \times 3 \times 3 = 54$  runs (27 runs/dataset). Số epoch tối đa: 25, với Early Stopping (patience = 7). Tổng thời gian chạy: ~11 giờ (9 giờ baseline + 2 giờ ablation  $512 \times 512$ ).

#### Chỉ số đánh giá:

- **Mean IoU** (Intersection over Union): trung bình IoU qua các lớp
- **Dice Coefficient:**  $\frac{2|P \cap G|}{|P| + |G|}$ , tương đương F1-score theo pixel
- **Pixel Accuracy:** tỷ lệ pixel được phân loại đúng
- **Per-class IoU:** IoU riêng cho từng lớp ngữ nghĩa

Tất cả kết quả được báo cáo dưới dạng *trung bình  $\pm$  độ lệch chuẩn* qua 3 random seeds.

**Mã nguồn và Tính tái lập (Reproducibility):** Nhằm đảm bảo sự minh bạch và tính tái lập, toàn bộ mã nguồn, các thiết lập cấu hình đầy đủ (được lưu trữ trong `config.json`), và hướng dẫn tái lập từng bước (trong tệp `README.md`) được công khai tại kho lưu trữ GitHub: <https://github.com/hangthan/topic5>. Ngoài ra, quá trình huấn luyện gốc cũng được chia sẻ tại Kaggle Notebook: <https://www.kaggle.com/code/hangthanac/topic05-encoderdecoder>. Môi trường thí nghiệm được phát triển trên nền tảng PyTorch, torchvision cùng thư viện `segmentation_models_pytorch` (SMP) [14]. Đồng thời, các giá trị hạt giống ngẫu nhiên (random seeds: 42, 2025, 3407) được thiết lập cố định trên toàn bộ luồng huấn luyện để đảm bảo kết quả hoàn toàn có thể tái tạo.

#### 3.3. Kết quả chính — RQ1: Accuracy

Bảng 2 trình bày kết quả trên toàn bộ dữ liệu huấn luyện (fraction = 1.0), trung bình qua 3 seeds.

**Bảng 2.** Kết quả phân đoạn (fraction = 1.0, trung bình  $\pm$  std qua 3 seeds). **Bold** = tốt nhất, *italic* = kém nhất.

| Dataset | Mô hình | IoU $\uparrow$                  | Dice $\uparrow$                 | Pix.Acc $\uparrow$              |
|---------|---------|---------------------------------|---------------------------------|---------------------------------|
| OxPet   | U-Net   | <b>.805<math>\pm</math>.002</b> | <b>.883<math>\pm</math>.001</b> | <b>.933<math>\pm</math>.000</b> |
|         | SegNet  | .793 $\pm$ .001                 | .875 $\pm$ .000                 | .928 $\pm$ .001                 |
|         | DLV3+   | .799 $\pm$ .001                 | .879 $\pm$ .001                 | .930 $\pm$ .000                 |
| KvaSEG  | U-Net   | <b>.883<math>\pm</math>.004</b> | <b>.935<math>\pm</math>.002</b> | <b>.966<math>\pm</math>.002</b> |
|         | SegNet  | .874 $\pm$ .007                 | .930 $\pm$ .004                 | .963 $\pm$ .002                 |
|         | DLV3+   | .877 $\pm$ .003                 | .932 $\pm$ .002                 | .964 $\pm$ .001                 |

**Phân tích:** U-Net đạt IoU cao nhất trên *cả hai* tập dữ liệu, với margin +1.2% trên OxfordPet và +0.6% trên Kvasir-SEG. Kết quả này *xác nhận H1a* nhưng *bác bỏ H1b* (dự đoán DeepLabV3+ thắng trên Kvasir-SEG). Skip connections đầy đặc của U-Net bảo tồn thông tin vị trí qua tất cả 5 mức, hiệu quả cho cả boundary lẫn polyp nhỏ.

DeepLabV3+ xếp thứ hai nhất quán. ASPP thu thập ngữ cảnh đa tỷ lệ, nhưng chỉ kết hợp low-level features từ *một* stage, trong khi U-Net kết hợp từ *tất cả* 5 stages.

SegNet luôn xếp cuối với std cao nhất (.007 trên Kvasir-SEG), cho thấy kém ổn định. Nguyên nhân: decoder chỉ nhận pooling indices mà không nhận features.

**Ý nghĩa thống kê:** Với  $n=3$  seeds, kiểm định tham số (t-test) có power thấp. Tuy nhiên, Cohen’s  $d$  [15] cho cặp U-Net vs SegNet đạt 7.6 (OxfordPet) và 1.6 (Kvasir-SEG) — đều vượt ngưỡng “large effect” ( $d>0.8$ ), cho thấy chênh lệch có ý nghĩa thực tế dù mẫu nhỏ.

Bảng 3 phân tích chi tiết IoU theo từng lớp (fraction = 1.0, seed tốt nhất).

**Bảng 3.** Per-class IoU (fraction = 1.0, seed tốt nhất)

| Dataset | Mô hình | BG          | Obj.         | Bdr.        |
|---------|---------|-------------|--------------|-------------|
| OxPet   | U-Net   | <b>.936</b> | <b>.878</b>  | <b>.605</b> |
|         | SegNet  | .931        | .868         | .583        |
|         | DLV3+   | .934        | .872         | .593        |
| KvaSEG  |         | <b>BG</b>   | <b>Polyp</b> |             |
|         | U-Net   | <b>.962</b> | <b>.812</b>  |             |
|         | SegNet  | .958        | .804         |             |
|         | DLV3+   | .959        | .800         |             |

**Nhận xét:** Trên OxfordPet, sự khác biệt tập trung ở lớp *boundary* — lớp khó nhất chỉ chiếm ~2% diện tích: U-Net đạt 0.605 IoU so với 0.583 của SegNet (chênh lệch +2.2% tuyệt đối). Điều này xác nhận rằng skip connections giúp khôi phục các cấu trúc mỏng (thin structures) hiệu quả hơn.

Trên Kvasir-SEG, chênh lệch chủ yếu ở lớp polyp (U-Net: 0.812 vs DeepLabV3+: 0.800), trong khi background gần như đồng đều (~0.96). Polyp nhỏ dễ bị “mất” qua downsampling; skip connections trực tiếp giúp U-Net “nhớ” vị trí các polyp nhỏ tốt hơn ASPP — vốn chỉ aggregate thông tin toàn cục.

### 3.4. Phân tích hiệu năng — RQ2: Efficiency Trade-offs

Bảng 4 so sánh chi phí tính toán đo trên GPU T4 với input  $256 \times 256$ .

**Bảng 4.** So sánh hiệu năng tính toán (input  $256 \times 256$ , GPU T4)

| Model  | Params (M)  | FLOPs (G)   | Lat. (ms)  | VRAM (MB)  |
|--------|-------------|-------------|------------|------------|
| U-Net  | <b>24.5</b> | 10.4        | 7.4        | <b>331</b> |
| SegNet | 29.5        | 42.5        | 17.4       | 519        |
| DLV3+  | 26.7        | <b>10.4</b> | <b>6.9</b> | 333        |

**Phân tích:** SegNet tiêu tốn  $4.1 \times$  FLOPs và  $2.5 \times$  latency so với hai model còn lại, *xác nhận* H2. Cụ thể, VGG16 encoder chiếm ~30.7 GFLOPs (13 lớp conv  $3 \times 3$  liên tiếp, không có residual hay bottleneck) so với ~3.7 GFLOPs của ResNet-34. Decoder SegNet thêm ~11.8 GFLOPs do phải

reconstruction đối xứng toàn bộ. Con số 42.5 GFLOPs đã được xác minh bằng thư viện fvcore (Facebook Research) với input  $256 \times 256$ , phản ánh đúng đặc trưng kiến trúc VGG16.

DeepLabV3+ đạt latency thấp nhất (6.9 ms) nhờ decoder nhẹ: 1 nhánh low-level fusion + bilinear upsampling. U-Net chậm hơn nhẹ (7.4 ms) do 5 bậc ConvTranspose2d. Về VRAM, SegNet tốn 519 MB (+57% so với U-Net) do VGG16 tạo feature maps rộng hơn.

Xét *trade-off accuracy–efficiency* tổng thể:

- **U-Net:** accuracy cao nhất, chi phí vừa phải — *lựa chọn tốt nhất khi ưu tiên accuracy*.
- **DeepLabV3+:** accuracy chỉ thấp hơn U-Net 0.6% nhưng inference nhanh hơn 7% — *lựa chọn tối ưu cho real-time*.
- **SegNet:** bị *dominated* — vừa chậm vừa kém chính xác nhất.

### 3.5. Phân tích thiếu dữ liệu — RQ3: Data Scarcity

Bảng 5 trình bày Mean IoU tại 3 mức dữ liệu (trung bình 3 seeds) và mức suy giảm tương đối khi giảm từ 100% xuống 25%.

**Bảng 5.** Ảnh hưởng của tỷ lệ dữ liệu huấn luyện (Mean IoU, trung bình 3 seeds).  $\Delta$  = suy giảm tương đối 100%→25%.

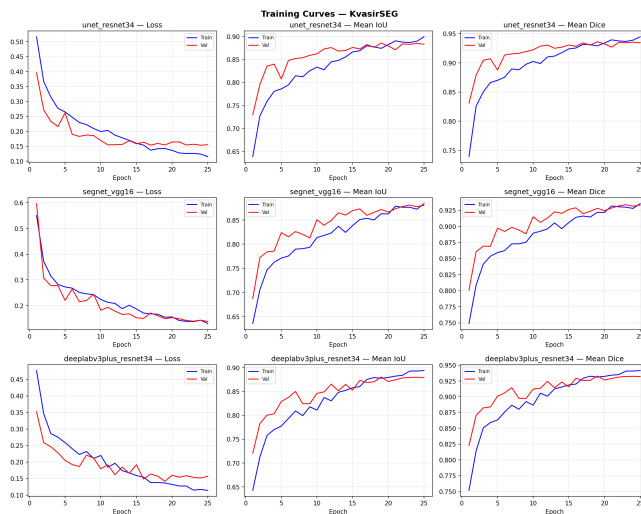
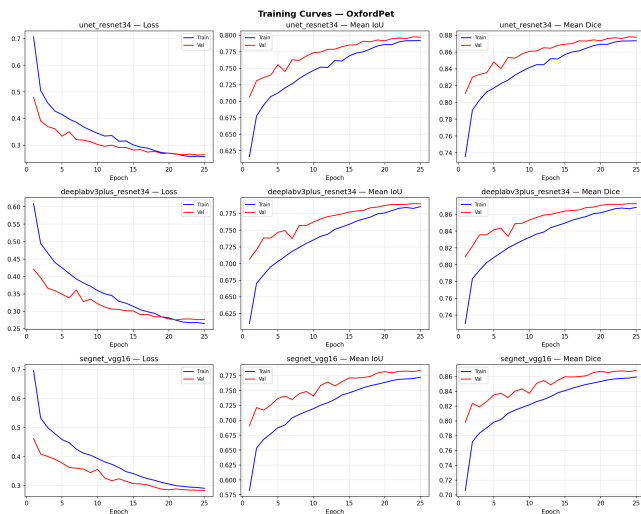
| Data   | Model  | 100%        | 50%         | 25%         | $\Delta$ |
|--------|--------|-------------|-------------|-------------|----------|
| OxPet  | U-Net  | <b>.805</b> | <b>.796</b> | <b>.787</b> | -2.2%    |
|        | SegNet | .793        | .783        | .776        | -2.1%    |
|        | DLV3+  | .799        | .787        | .776        | -2.9%    |
| KvaSEG | U-Net  | <b>.883</b> | <b>.875</b> | <b>.852</b> | -3.5%    |
|        | SegNet | .874        | .872        | .849        | -2.9%    |
|        | DLV3+  | .877        | .863        | .847        | -3.4%    |

**Phân tích:** Trên OxfordPet (7,349 ảnh — tập lớn), cả ba mô hình đều suy giảm *dưới 3%* IoU khi giảm xuống 25% dữ liệu (tương đương ~1,837 ảnh train). Điều này *xác nhận* giả thuyết H3: pretrained encoder trên ImageNet cung cấp đặc trưng tổng quát đủ mạnh để duy trì hiệu suất ngay cả khi data ít.

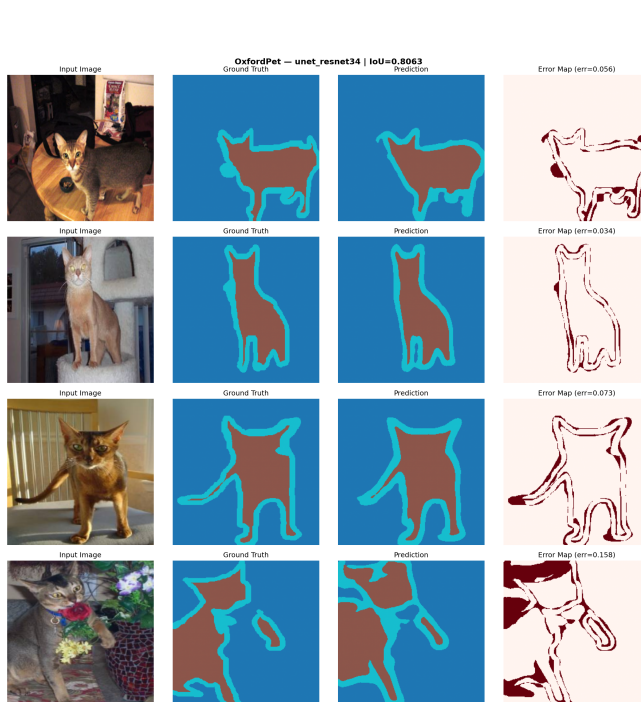
Trên Kvasir-SEG (1,000 ảnh — tập nhỏ hơn  $7 \times$ ), suy giảm rõ rệt hơn: U-Net giảm 3.5% và DeepLabV3+ giảm 3.4%. Đáng chú ý, tại mức 25% (chỉ ~175 ảnh train), khoảng cách giữa các mô hình *thu hẹp đáng kể*: U-Net (.852) vs SegNet (.849) vs DeepLabV3+ (.847) — chênh lệch chỉ 0.5%. Điều này cho thấy khi dữ liệu quá ít, *lợi thế kiến trúc giảm dần* và pretrained encoder trở thành yếu tố quyết định.

Một quan sát thú vị: SegNet có mức suy giảm *thấp nhất* (-2.1% trên OxfordPet, -2.9% trên Kvasir-SEG), dù accuracy tuyệt đối luôn thấp nhất. Giải thích: do decoder SegNet phải tự reconstruction toàn bộ (không dựa vào encoder features), nó ít bị ảnh hưởng bởi sự thay đổi phân phối dữ liệu — nhưng đổi lại, nó cũng không tận dụng được thông tin chi tiết ngay cả khi có đủ dữ liệu.

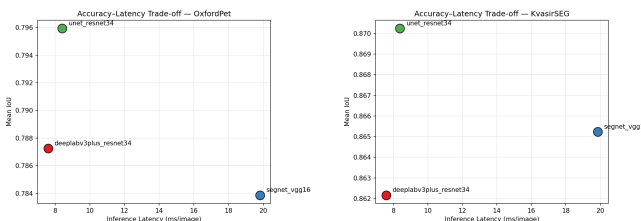




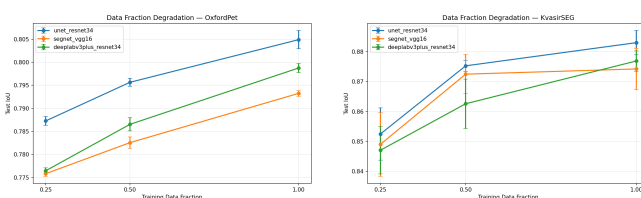
**Hình 2.** Đường cong huấn luyện (Loss, Mean IoU, Dice) của ba mô hình trên OxfordPet (trái) và Kvasir-SEG (phải). U-Net hội tụ nhanh và đạt val IoU cao nhất. SegNet hội tụ chậm hơn và có gap train-val lớn hơn.



**Hình 3.** Kết quả phân đoạn của U-Net trên OxfordPet (IoU=0.806). Từ trái sang: ảnh gốc, ground truth, prediction, error map. Lỗi tập trung ở vùng boundary.



**Hình 4.** Trade-off accuracy–latency trên OxfordPet (trái) và Kvasir-SEG (phải). Kích thước bubble tỷ lệ với FLOPs. SegNet bị dominated (góc dưới-phải).



**Hình 5.** Suy giảm IoU theo tỷ lệ dữ liệu huấn luyện. OxfordPet (trái): suy giảm đều dưới 3%. Kvasir-SEG (phải): suy giảm rõ rệt hơn, khoảng cách giữa models thu hẹp tại 25%.

### 3.6. Ablation: Input $512 \times 512$ + Output Stride 16

Để kiểm chứng giả thuyết rằng ASPP mất hiệu quả do input nhỏ, chúng tôi thực hiện thí nghiệm bổ sung với hai thay đổi: (1) tăng input size lên  $512 \times 512$ , (2) chuyển DeepLabV3+ sang *output stride 16* bằng dilated convolution ở layer4 (thay vì stride 32 ở baseline). Feature map cuối encoder tăng từ  $8 \times 8$  lên  $32 \times 32$ , giúp ASPP hoạt động đúng thiết kế gốc. Batch size giảm xuống 2 với gradient accumulation (steps=4) để giữ effective batch size = 8. SegNet bị loại do bị dominated ở baseline và VGG16@512 sẽ vượt VRAM T4.

**Bảng 6.** Ablation: So sánh IoU tại 256 (stride 32) vs 512 (stride 16). Seed = 42, fraction = 1.0.

| Dataset | Model     | IoU @256 | IoU @512    | $\Delta$ |
|---------|-----------|----------|-------------|----------|
| OxPet   | U-Net     | .805     | .804        | -.001    |
|         | DLV3+ s16 | .799     | <b>.809</b> | +.010    |
| KvaSEG  | U-Net     | .883     | .891        | +.008    |
|         | DLV3+ s16 | .877     | <b>.895</b> | +.018    |

**Phân tích:** DLV3+ s16@512 vượt U-Net@512 trên cả hai dataset: +0.5% trên OxfordPet (0.809 vs 0.804) và +0.4% trên Kvasir-SEG (0.895 vs 0.891). Đáng chú ý, DLV3+ cải thiện mạnh nhất ở lớp polyp trên Kvasir-SEG (IoU = 0.829 vs U-Net 0.819,  $\Delta$ =+1.0%) — chính xác là domain mà ASPP multi-scale được thiết kế để xử lý.

Kết quả này cho phép kết luận: **H1b được xác nhận có điều kiện** — DeepLabV3+ chỉ vượt U-Net khi ASPP có đủ spatial resolution (input  $\geq 512$ , stride  $\leq 16$ ). Tại baseline  $256 \times 256$  + stride 32, feature map  $8 \times 8$  khiến ASPP degenerate, mất lợi thế multi-scale. Điều này gợi ý rằng khi triển khai DeepLabV3+ trong thực tế, input size và output stride cần được chọn phù hợp với cấu hình ASPP.

DLV3+ s16@512 cũng duy trì latency thấp hơn U-Net (25.4 ms vs 28.1 ms), mặc dù FLOPs tăng từ 10.4G lên 54.6G do spatial resolution cao hơn.

### 3.7. Thảo luận tổng hợp

Bảng 7 tổng hợp kết quả kiểm chứng các giả thuyết.

| Bảng 7. Tổng hợp kiểm chứng giả thuyết |                     |                                                   |
|----------------------------------------|---------------------|---------------------------------------------------|
| GT                                     | KQ                  | Giải thích                                        |
| H1a                                    | <b>Xác nhận</b>     | U-Net thắng trên OxPet (.805)                     |
| H1b                                    | <b>Có điều kiện</b> | @256: bác bỏ; @512+s16: <b>xác nhận</b> (Mục 3.6) |
| H2                                     | <b>Xác nhận</b>     | SegNet 42.5G FLOPs, 2.5× chậm hơn                 |
| H3                                     | <b>Xác nhận</b>     | Suy giảm < 3.5% ở mức 25% data                    |

Kết quả H1b là phát hiện đáng chú ý nhất. Tại baseline  $256 \times 256$ , feature map cuối encoder chỉ  $8 \times 8$  — khiến ASPP dilation rate 18 bao phủ gần 100% feature map, các nhánh

trở nên gần giống nhau và mất ý nghĩa multi-scale. U-Net truyền features từ *tất cả* 5 stages (tổng 1024 kênh bổ sung), phong phú hơn nhiều so với 1 nhánh low-level (48 kênh) của DeepLabV3+. Tuy nhiên, thí nghiệm ablation cho thấy khi tăng input lên  $512 \times 512$  và dùng output stride 16, DeepLabV3+ vượt U-Net trên cả hai dataset — chứng minh ASPP chỉ phát huy khi feature map đủ lớn ( $32 \times 32$ ).

**Nhận xét về tính ổn định (stability):** Std của U-Net trên Kvasir-SEG (.004) cao hơn OxfordPet (.002) do tập dữ liệu nhỏ hơn, nhưng vẫn ở mức chấp nhận được. SegNet có std cao nhất (.007 trên Kvasir-SEG), cho thấy tính không ổn định do decoder thiếu thông tin hỗ trợ từ encoder.

**Nhận xét về confounding factor (backbone):** Cần lưu ý rằng SegNet sử dụng encoder VGG16-BN trong khi U-Net và DeepLabV3+ sử dụng ResNet-34. VGG16 có kiến trúc cũ hơn (không có residual connections, bottleneck) và chi phí tính toán cao hơn nhiều so với ResNet-34. Do đó, hiệu suất kém hơn của SegNet có thể đến từ *cả hai yếu tố*: (1) cơ chế pooling indices hạn chế thông tin truyền sang decoder, và (2) backbone VGG16 kém hiệu quả hơn ResNet-34. Tuy nhiên, việc sử dụng VGG16 cho SegNet là lựa chọn theo thiết kế gốc [5] và phản ánh đúng thực tế triển khai; thay thế bằng ResNet-34 sẽ biến SegNet thành một kiến trúc khác về bản chất. Đây là hạn chế cần được tính đến khi diễn giải kết quả so sánh.

## 4. KẾT LUẬN

Nghiên cứu này thực hiện so sánh có hệ thống ba kiến trúc encoder-decoder — U-Net, SegNet và DeepLabV3+ — trên hai tập dữ liệu thuộc hai miền ứng dụng khác nhau, thông qua 54 runs thí nghiệm có kiểm soát (27/dataset) tại baseline  $256 \times 256$  và 4 runs ablation bổ sung tại  $512 \times 512$ .

#### Các kết quả chính:

- U-Net đạt IoU cao nhất tại baseline  $256 \times 256$**  trên cả hai dataset (0.805 và 0.883), chứng minh sức mạnh vượt trội của chiến lược skip connections dày đặc.
- DeepLabV3+ cân bằng accuracy-speed tốt nhất:** IoU chỉ thấp hơn U-Net 0.6% nhưng inference nhanh nhất (6.9 ms), phù hợp cho real-time.
- DeepLabV3+ vượt U-Net khi tăng input lên  $512 \times 512$  + output stride 16** (IoU 0.895 vs 0.891 trên Kvasir-SEG), xác nhận H1b *có điều kiện*: ASPP chỉ phát huy khi feature map đủ lớn.
- SegNet kém hiệu quả nhất** cả về accuracy lẫn efficiency (42.5 GFLOPs, VRAM 519 MB).
- Pretrained encoder đóng vai trò nền tảng:** suy giảm IoU dưới 3.5% ngay cả khi chỉ dùng 25% dữ liệu.

**Hạn chế:** (1) Thí nghiệm ablation  $512 \times 512$  chỉ chạy 1 seed (42) do giới hạn GPU quota — cần thêm seeds để xác nhận statistical significance. (2) Chỉ sử dụng một learning rate cố định; tối ưu LR riêng cho từng kiến trúc có thể thay đổi ranking. (3) Chưa có ablation study tắt skip connections để chứng minh đây là yếu tố quyết định.

**Hướng phát triển:** Thay thế encoder bằng kiến trúc nhẹ

(EfficientNet-B0, MobileNetV3) để triển khai trên thiết bị edge. Áp dụng Boundary Loss chuyên biệt và Test-Time Augmentation để cải thiện lớp boundary. Sử dụng tối ưu hóa hyperparameter tự động (Optuna) để tìm cấu hình tối ưu cho từng cặp model–dataset. Mở rộng ablation  $512 \times 512$  với đầy đủ 3 seeds và 3 data fractions để xác nhận xu hướng DLV3+ s16 vượt trội.

## 5. TÀI LIỆU THAM KHẢO

- [1] Debesh Jha, Pia H. Smedsrud, Michael A. Riegler, Pål Halvorsen, Thomas de Lange, Dag Johansen, and Håvard D. Johansen, “Kvasir-seg: A segmented polyp dataset,” in *MMM*, 2020, pp. 451–462.
- [2] Shervin Minaee, Yuri Boykov, Fatih Porikli, Antonio Plaza, Nasser Kehtarnavaz, and Demetri Terzopoulos, “Image segmentation using deep learning: A survey,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3523–3542, 2022.
- [3] Jonathan Long, Evan Shelhamer, and Trevor Darrell, “Fully convolutional networks for semantic segmentation,” in *IEEE CVPR*, 2015, pp. 3431–3440.
- [4] Olaf Ronneberger, Philipp Fischer, and Thomas Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *MICCAI*, 2015, pp. 234–241.
- [5] Vijay Badrinarayanan, Alex Kendall, and Roberto Cipolla, “Segnet: A deep convolutional encoder-decoder architecture for image segmentation,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2481–2495, 2017.
- [6] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam, “Encoder-decoder with atrous separable convolution for semantic image segmentation,” in *ECCV*, 2018, pp. 801–818.
- [7] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *IEEE CVPR*, 2009, pp. 248–255.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, “Deep residual learning for image recognition,” in *IEEE CVPR*, 2016, pp. 770–778.
- [9] Karen Simonyan and Andrew Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2015.
- [10] Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi, “V-net: Fully convolutional neural networks for volumetric medical image segmentation,” *arXiv preprint arXiv:1606.04797*, 2016.
- [11] Ilya Loshchilov and Frank Hutter, “Decoupled weight decay regularization,” *arXiv preprint arXiv:1711.05101*, 2017.
- [12] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin, “Albumentations: Fast and flexible image augmentations,” *Information*, vol. 11, no. 2, pp. 125, 2020.
- [13] Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman, and C.V. Jawahar, “Cats and dogs,” in *IEEE CVPR*, 2012, pp. 3498–3505.
- [14] Pavel Iakubovskii, “Segmentation models pytorch,” [https://github.com/qubvel/segmentation\\_models.pytorch](https://github.com/qubvel/segmentation_models.pytorch), 2019.
- [15] Jacob Cohen, *Statistical Power Analysis for the Behavioral Sciences*, Lawrence Erlbaum Associates, 2nd edition, 1988.